

ELEC5803 HLS Exam Study

Step-by-Step Solution to Question 1

Course-slide basis

This solution follows the same HLS flow used in the course slides:

- **Slides 5:** *Behavioral HDL to CDFG Translation or Compilation, ASAP and ALAP Scheduling and Mobility*, and resource-constrained scheduling in `Slides/ELEC5803_Slides5_HLS Algorithms_Part1.pdf`.
- **Slides 6:** scheduling versus binding/allocation, datapath synthesis, and compatibility-based sharing in `Slides/ELEC5803_Slides6_HLS Algorithms_Part2.pdf`.
- **Slides 7:** FSM/FSMD-style controller realization in `Slides/ELEC5803_Slides7_Control Unit Design_AA1.pdf`.

Important interpretation note

The printed VHDL in the sample exam is clearly intended to implement **Stein's algorithm** (binary GCD), but it is not fully self-consistent as written:

- the port declaration is syntactically incomplete,
- the “both-even” case of Stein's algorithm is missing,
- `shift_count` is declared but never incremented,
- the final result should be restored by a left shift, i.e. multiplying by 2^k .

Therefore, the solution below uses the **intended binary-GCD algorithm**, which is the only interpretation consistent with the comments in the code and with the exam questions on flowchart, DFG, scheduling, and hardware design.

1. Intended algorithm

Let k denote the number of common factors of 2 removed from both operands.

1. Load $x \leftarrow X_IN$, $y \leftarrow Y_IN$, $k \leftarrow 0$.
2. If $x = 0$, output y . If $y = 0$, output x .
3. Repeat until $x = y$:
 - If x and y are both even, set $x \leftarrow x/2$, $y \leftarrow y/2$, $k \leftarrow k + 1$.
 - Else if x is even, set $x \leftarrow x/2$.
 - Else if y is even, set $y \leftarrow y/2$.

- Else if $x > y$, set $x \leftarrow (x - y)/2$.
- Else set $y \leftarrow (y - x)/2$.

4. When $x = y$, output $\text{GCD} = x \ll k$.

2. Flowchart of the algorithm

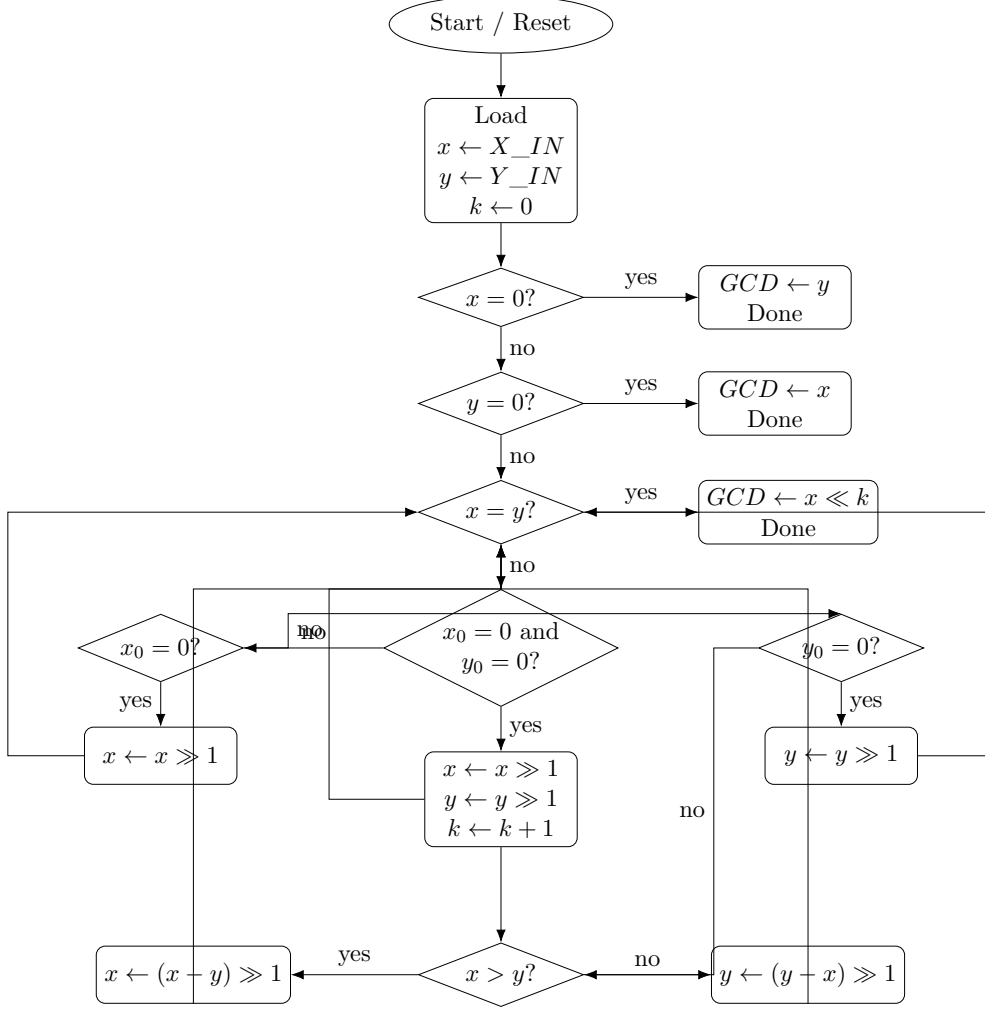


Figure 1: Flowchart for the intended Stein/binary-GCD algorithm. This is the natural control-flow representation corresponding to the behavior-to-CDFG discussion in Slides 5.

3. Data-flow graph (DFG) / sequencing view

To match the style used in the course slides, it is better to separate:

- an **unscheduled sequencing DFG**, which only shows operations and precedence edges, and
- a **scheduled DFG**, which groups the same operations into control steps.

This is the same presentation logic used in the Slides-5 examples around the HAL DFG and the scheduled DFG.

For one non-terminal iteration, the main arithmetic candidates are:

$$v_1 : s_x = x \gg 1$$

$$v_2 : s_y = y \gg 1$$

$$v_3 : k_1 = k + 1$$

$$v_4 : d_{xy} = x - y$$

$$v_5 : d_{yx} = y - x$$

$$v_6 : n_x = d_{xy} \gg 1$$

$$v_7 : n_y = d_{yx} \gg 1$$

$$v_8 : g = x \ll k$$

with dependencies $v_4 \rightarrow v_6$ and $v_5 \rightarrow v_7$.

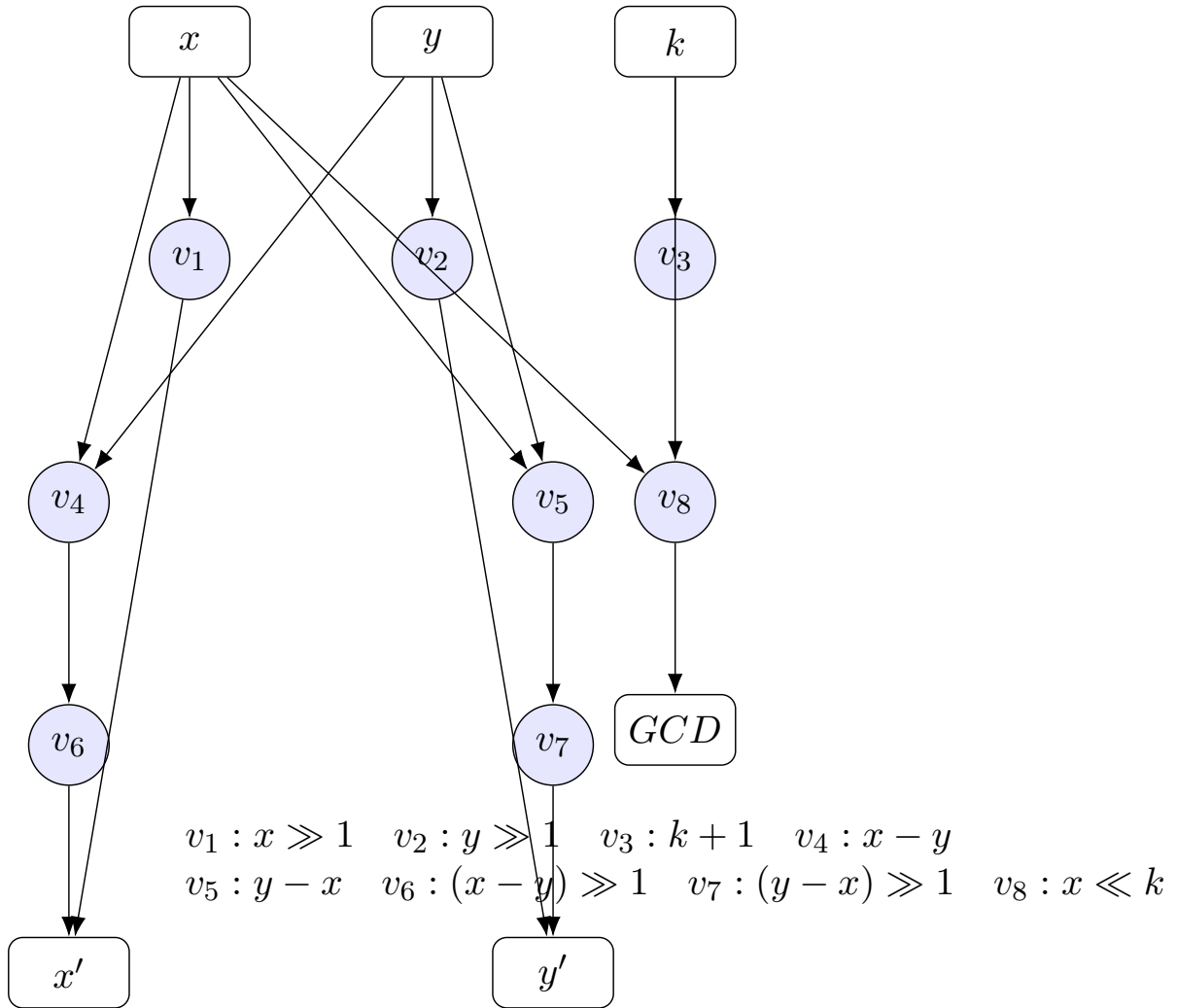


Figure 2: Unscheduled sequencing DFG for one Stein-algorithm iteration, drawn in the same operation-vertex style as the DFG figures in the course slides.

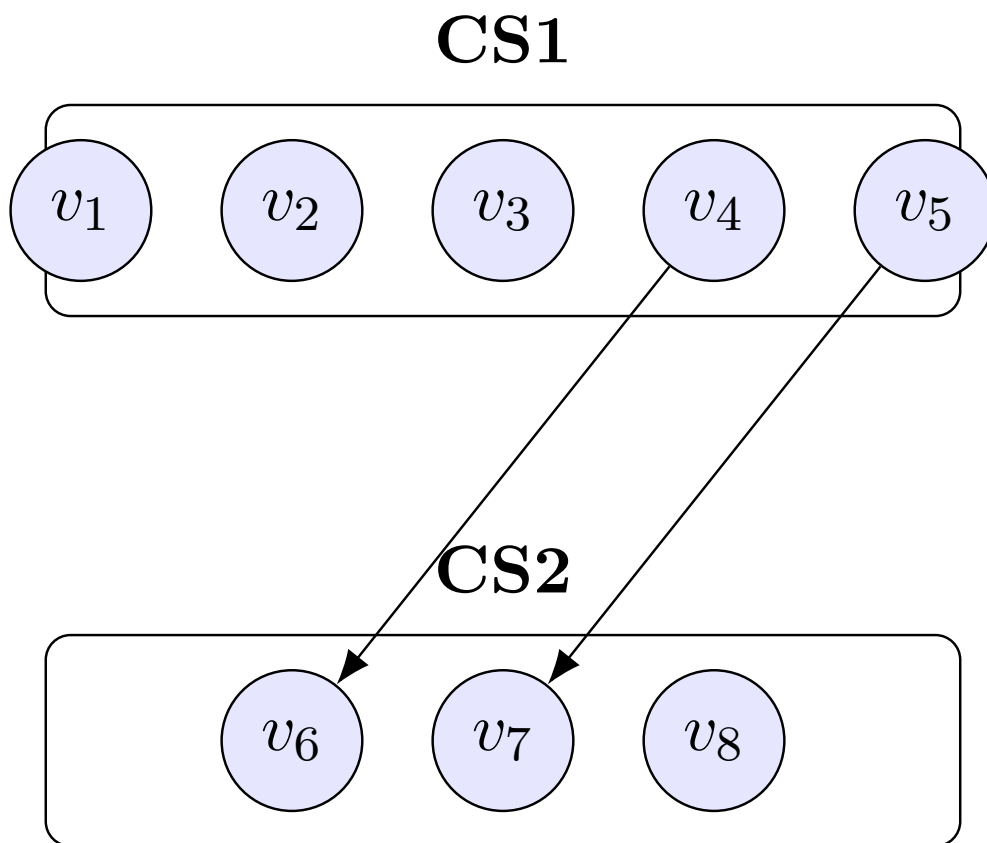


Figure 3: Scheduled DFG in slide style: the same operation vertices grouped by control step. This corresponds to the ASAP schedule used in part (c).

3.1 Controller FSM for the minimum-area FSMD

To match the control-unit style from Slides 7, the datapath can be driven by a Moore-style FSM whose states correspond to the main algorithm phases.

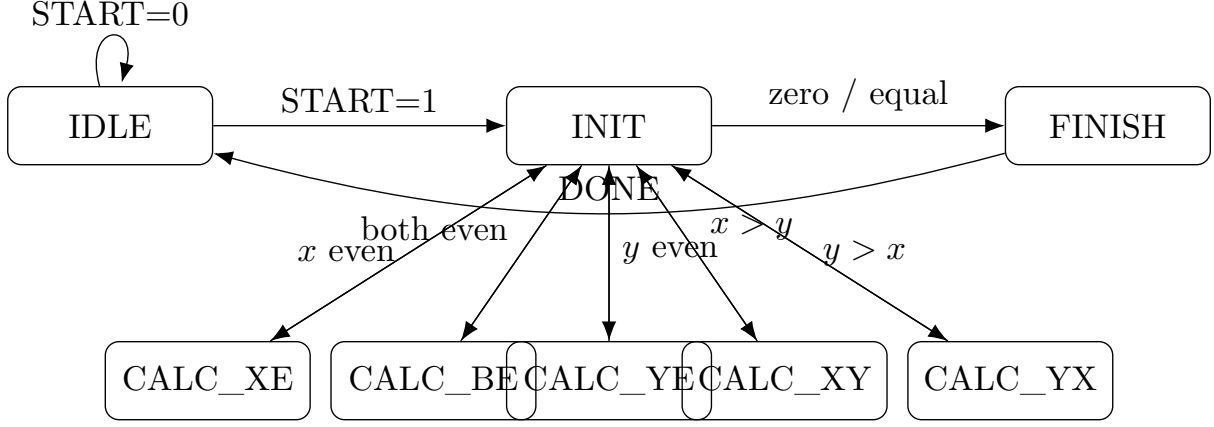


Figure 4: State-transition graph for the controller FSM. This is the same kind of FSM/STG representation used in the control-unit slides.

Present state	Condition	Next state	Moore-state action / micro-operation
IDLE	START=0	IDLE	Wait; DONE=0
IDLE	START=1	INIT	Load $x \leftarrow X_IN$, $y \leftarrow Y_IN$, $k \leftarrow 0$
INIT	$x = 0$	FINISH	$GCD \leftarrow y$
INIT	$y = 0$	FINISH	$GCD \leftarrow x$
INIT	$x = y$	FINISH	$GCD \leftarrow x \ll k$
INIT	$x_0 = 0 \wedge y_0 = 0$	CALC_BE	Select both-even path
INIT	$x_0 = 0 \wedge y_0 = 1$	CALC_XE	Select x -even path
INIT	$x_0 = 1 \wedge y_0 = 0$	CALC_YE	Select y -even path
INIT	$x_0 = 1 \wedge y_0 = 1 \wedge x > y$	CALC_XY	Select odd-odd, $x > y$ path
INIT	$x_0 = 1 \wedge y_0 = 1 \wedge y > x$	CALC_YX	Select odd-odd, $y > x$ path
CALC_BE	unconditional	INIT	$x \leftarrow x \gg 1$, $y \leftarrow y \gg 1$, $k \leftarrow k + 1$
CALC_XE	unconditional	INIT	$x \leftarrow x \gg 1$
CALC_YE	unconditional	INIT	$y \leftarrow y \gg 1$
CALC_XY	unconditional	INIT	$x \leftarrow (x - y) \gg 1$
CALC_YX	unconditional	INIT	$y \leftarrow (y - x) \gg 1$
FINISH	unconditional	IDLE	DONE=1

The table above plays the same role as the *state transition table / direct structure table* in the FSM slides: it lists present state, transition condition, next state, and the state-associated actions.

4. ASAP and ALAP scheduling

Slides 5 define:

- **ASAP**: place each operation in the earliest possible control step.
- **ALAP**: place each operation in the latest possible control step without increasing the latency obtained from ASAP.
- **Mobility**: $\text{ALAP} - \text{ASAP}$.

Here we schedule the arithmetic DFG vertices $\{v_1, \dots, v_8\}$ with the standard simplifying assumption used in scheduling exercises:

- each arithmetic operation takes one control step,
- predicate evaluation belongs to the controller,
- unlimited resources are available for parts (c) ASAP/ALAP only.

4.1 Precedence constraints

Only two true arithmetic dependencies exist inside the iteration kernel:

$$v_4 \rightarrow v_6, \quad v_5 \rightarrow v_7$$

All of the other candidate results are produced directly from the current-state registers.

4.2 ASAP schedule

The minimum achievable latency for this DFG is therefore **2 control steps**.

Operation	CS1	CS2
$v_1 : x \gg 1$	•	
$v_2 : y \gg 1$	•	
$v_3 : k + 1$	•	
$v_4 : x - y$	•	
$v_5 : y - x$	•	
$v_6 :$ $(x - y) \gg 1$		•
$v_7 :$ $(y - x) \gg 1$		•
$v_8 : x \ll k$	•	

4.3 ALAP schedule

Using the same total latency of 2 steps:

Operation	CS1	CS2
$v_1 : x \gg 1$		•
$v_2 : y \gg 1$		•
$v_3 : k + 1$		•
$v_4 : x - y$	•	
$v_5 : y - x$	•	
$v_6 :$ $(x - y) \gg 1$		•
$v_7 :$ $(y - x) \gg 1$		•
$v_8 : x \ll k$		•

4.4 Mobility table

Vertex	ASAP	ALAP	Mobility
v_1	1	2	1
v_2	1	2	1
v_3	1	2	1
v_4	1	1	0
v_5	1	1	0
v_6	2	2	0
v_7	2	2	0
v_8	1	2	1

4.5 Interpretation

This result matches the course discussion in Slides 5:

- subtraction nodes v_4 and v_5 have **zero mobility** because their shifted successors force them into control step 1,
- shifts on the current operands and the final restoration shift have **mobility 1**,
- this mobility information is exactly what later guides hardware sharing and allocation.

5. Hardware design with minimum area

For minimum area, the correct HLS strategy from Slides 6 is to **maximize hardware sharing** and allow a multi-cycle iterative implementation.

5.1 Datapath idea

Use one shared arithmetic datapath:

- registers: R_x , R_y , R_k , and an output register R_g ,
- one shared ALU to perform subtraction and increment,
- one shared shifter to perform right shifts and the final left shift,
- one comparison/status block producing $x = 0$, $y = 0$, $x = y$, $x > y$, x_0 , and y_0 ,
- multiplexers at ALU and shifter inputs,
- one FSM controller.

5.2 Why this is minimum-area

The hardware is area-efficient because:

- only **one arithmetic unit** is used for all subtract and increment operations,
- only **one shifter** is used for all divide-by-2 and final restore operations,
- the algorithm is iterative, so throughput is sacrificed in exchange for hardware reuse,
- the controller sequences the shared datapath exactly as in the FSMD model from Slides 7.

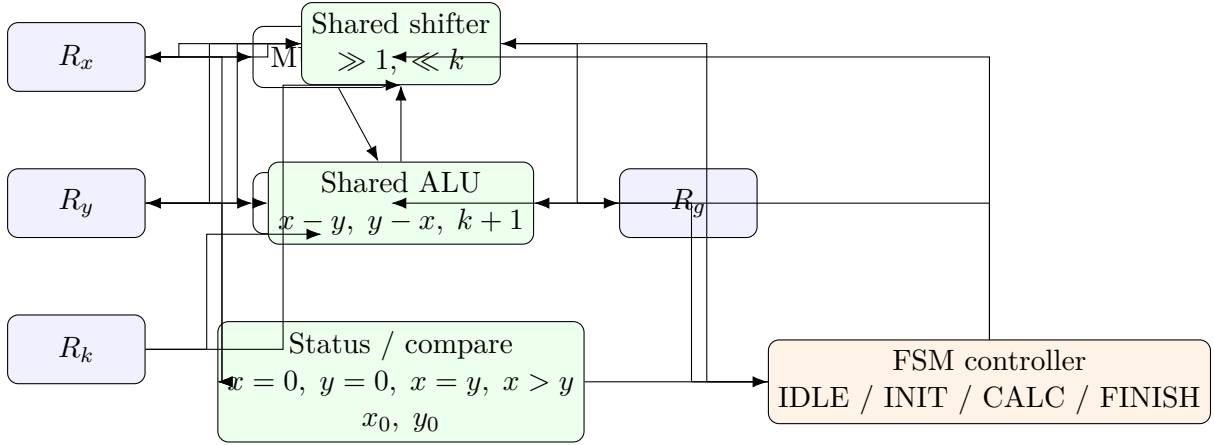


Figure 5: Minimum-area FSMD-style hardware. The same ALU and shifter are reused across multiple control steps, which is the key Slides-6 allocation idea for minimizing area.

5.3 Typical operation sequence

One possible control realization is:

State	Action
IDLE	Wait for START=1 .
INIT	Load $R_x \leftarrow X_IN$, $R_y \leftarrow Y_IN$, $R_k \leftarrow 0$; check zero cases.
CALC_BE	Both-even case: use shifter twice (or in two successive micro-operations) to update x and y , then ALU for $k + 1$.
CALC_XE	x even: shift x right by one.
CALC_YE	y even: shift y right by one.
CALC_XY	Odd-odd and $x > y$: ALU computes $x - y$, then shifter computes $(x - y) \gg 1$, then write back to R_x .
CALC_YX	Odd-odd and $y > x$: ALU computes $y - x$, then shifter computes $(y - x) \gg 1$, then write back to R_y .
FINISH	Use shifter for $x \ll k$, store to R_g , assert DONE .

This design uses the **fewest functional units**, but it needs the **largest number of control steps per iteration**.

6. Hardware design with minimum latency

For minimum latency, the HLS goal changes: instead of maximizing reuse, we allocate more hardware so that as much work as possible is done in parallel during each loop iteration.

6.1 Datapath idea

Compute all candidate next values in parallel:

- one right shifter for $x \gg 1$,
- one right shifter for $y \gg 1$,
- one subtractor for $x - y$ followed by a shifter for $(x - y) \gg 1$,
- one subtractor for $y - x$ followed by a shifter for $(y - x) \gg 1$,
- one incrementer for $k + 1$,
- one left shifter for the final $x \ll k$,
- parallel comparators for all branch conditions,
- multiplexers that select the correct next-state values.

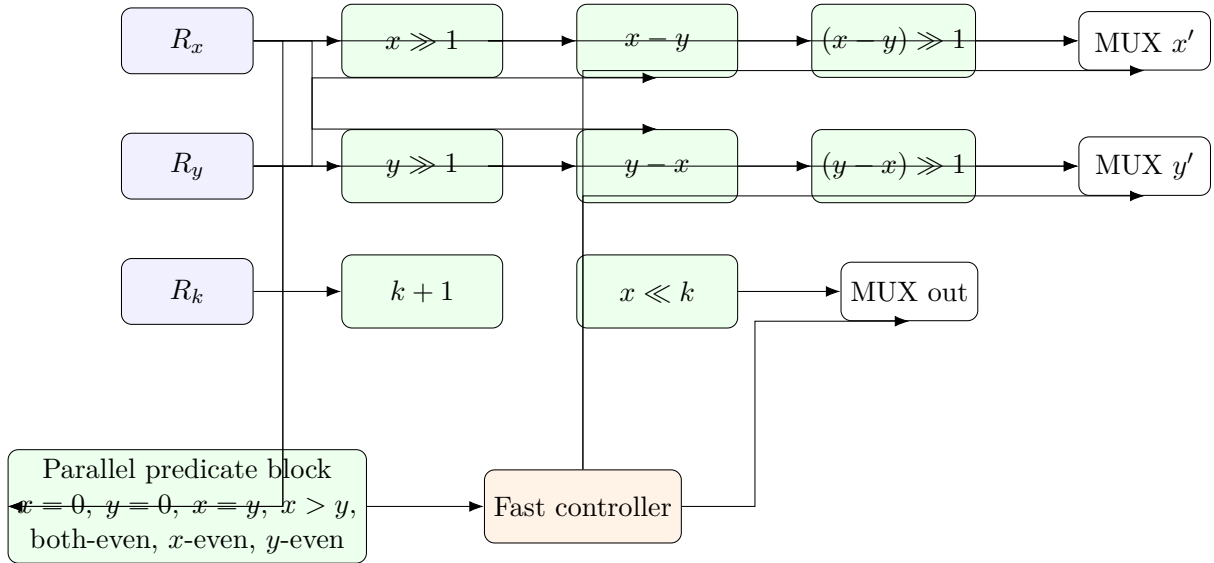


Figure 6: Conceptual minimum-latency organization: all candidate branch results are computed in parallel and a controller/multiplexer network commits the correct next values.

6.2 Minimum-latency architecture in words

Because the algorithm has a **loop-carried dependency**, we cannot eliminate the loop itself. Therefore, “minimum latency” means:

- **minimum control steps per iteration**, ideally one cycle per iteration, and
- parallel computation of all branch candidates so the next-state decision is only a mux selection.

The datapath can therefore update the state in one iteration-cycle as follows:

- if both inputs are even, load $x' = x \gg 1$, $y' = y \gg 1$, and $k' = k + 1$ simultaneously;
- if only one operand is even, shift that operand in the same cycle;
- if both are odd, compute both $x - y$ and $y - x$ in parallel and select the correct shifted result using the $x > y$ comparator;
- when $x = y$, produce $x \ll k$ immediately through the output shifter.

6.3 Functional-unit count

A natural minimum-latency allocation is:

Functional unit	Count
Subtractors	2
Right shifters	4
Incrementer	1
Left shifter	1
Comparators / parity detectors	parallel status block
State/output registers	4

This is not the minimum-area design, but it minimizes execution time by removing most structural conflicts.

7. Final exam-style answers

(a) Flowchart

Use the flowchart in Figure 1. It is based on the standard Stein algorithm with zero checks, equality check, both-even case, single-even cases, and odd-odd subtraction cases.

(b) Data flowgraph

Use the slide-style sequencing DFG and scheduled DFG in Figures 2 and 3. The controller FSM and its table are given immediately after the DFG, following the same state-graph/table pattern used in the control-unit slides.

(c) ASAP and ALAP scheduling

With one-cycle arithmetic operations and unlimited resources:

- ASAP latency is 2 control steps.
- ALAP latency is also 2 control steps.
- Zero-mobility nodes are v_4 , v_5 , v_6 , and v_7 .

(d) Minimum-area hardware

Choose a shared iterative FSM:

- one ALU,
- one shifter,
- one status/comparator block,

- registers for x , y , k , and output,
- FSM controller.

This minimizes the number of functional units, but increases the number of cycles.

(e) Minimum-latency hardware

Choose a parallel datapath:

- parallel subtractors, shifters, incrementer, and status logic,
- branch candidates computed concurrently,
- multiplexer-based commit of the correct next-state values.

This minimizes the number of cycles per iteration, but increases hardware area.

8. Conclusion

The overall HLS reasoning matches the lecture flow:

behavior $\rightarrow$ CDFG/DFG $\rightarrow$ scheduling $\rightarrow$ allocation/binding $\rightarrow$ FSMD

which is exactly the sequence emphasized across Slides 5–7.